

Architectures and Platforms for AI

Module 2

Lorenzo Pellegrino

November 8, 2025

Contents

1 Linear Classifier

1.1 Definition

A linear classifier is described by the equation $f(x, W) = Wx + b$, where:

- $x \in \mathbf{R}^d$ is the input vector, a single observation living in a d -dimensional space where each coordinate represents a measured feature. In geometric terms, x is a point in the landscape the classifier must learn to partition.
- $W \in \mathbf{R}^{d \times k}$ contains the model's parameters. Each column defines a direction in feature space associated with one class: together they form k geometric “templates” the model uses to evaluate similarity. Changing W reshapes the orientation of the decision hyperplanes, dictating how sensitive the classifier is to each input dimension. In practice it measures how much each of the d features of the input contributes to the construction of the score for each of the possible k classes.
- $b \in \mathbf{R}^k$ is the bias vector, translating those hyperplanes through space. It decouples the classification boundaries from the origin, allowing the model to represent structure that does not naturally center around zero—a small but essential freedom for any realistic dataset.

1.2 Training

When we train the classifier we want to find the best weights and biases so that the model best represents all the classes. In order to evaluate the classifier there is the concept of loss.

A loss L_i is a function that, considering a known dataset (x_i, y_i) with $i \in [1, N]$ where for each input x_i there is a known ground truth class y_i , computes the difference between the prediction and the true label. It is averaged over all the data in the dataset. Moreover regularization is applied: we give a weight λ to how $R(\cdot)$ interprets a complexity of the model which is represented by W . In this way we penalize the model from overfitting the training data.

$$L = \frac{1}{N} \sum_i L_i(f_i(x, W), y_i) + \lambda R(W)$$

A major limitation of the linear classifier is that it can only separate data using linear hyperplanes. This constraint makes it ineffective when classes are not linearly separable in the input space, for instance, attempting to classify points arranged in a circular pattern with a straight line is geometrically impossible.

To overcome this, the **kernel trick** maps the input data into a higher-dimensional feature space through a kernel function, in which the samples become linearly separable and a linear classifier can operate effectively. Different kernels induce different feature spaces and levels of model complexity. However, higher flexibility comes with the risk of

overfitting: as the kernel becomes more expressive, it may fit noise and lose generalization on unseen data, especially in large or noisy datasets (capacity–generalization trade-off).

1.3 Feature selection

Before feeding data into the fully connected (FC) layer, they must be **preprocessed** to remove noise and highlight the most informative features, thereby improving classification performance.

This stage involves two key operations: **feature extraction**, which constructs new, more expressive features from existing ones, and **feature selection**, which identifies a subset of the most relevant original features. Both methods aim to reduce dimensionality, eliminating variables that contribute little useful information or that may introduce redundancy and noise.

By focusing on the most discriminative aspects of the data, the model becomes more efficient, less prone to overfitting, and easier to generalize to unseen samples (e.g. principle component analysis or PCA).

Before the exploding use of deep neural networks (DNNs), these operations were done manually or with shallow statistical models. Anyway they had many issues:

1. **Manual feature design doesn't scale**

Handcrafted features worked for simple, low-dimensional tasks but failed when the structure of the data was intricate or abstract.

2. **Methods assumed linear or low-order relationships**

Real-world data, however, often contains highly nonlinear dependencies that linear combinations or simple transformations can't express.

3. **Feature engineering locked learning into a human bias**

DNNs changed this dynamic by learning hierarchical representations directly from raw data. Lower layers capture simple patterns, and successive layers combine them into complex abstractions. This automatic feature learning removed the need for explicit handcrafted preprocessing and allowed models to adapt their internal representation to the task, yielding far better performance on unstructured, high-dimensional data.

Training a deep network means not only learning to classify data, but also learning the representations that make such classification possible.

2 Deep Neural Networks

2.1 Definition

DNNs are fundamentally extensions of the linear perceptron. It is the simplest type of artificial neuron, a computational model inspired by a biological neuron. It can only classify the input vectors into two categories (binary classification).

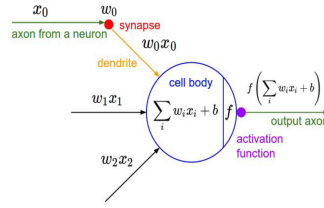


Figure 1: Linear perceptron vs Neuron

The DNNs then are built by composing multiple perceptrons in layers (input, hidden, output) and then applying non-linear activation functions at each neuron so the network can approximate complex, non-linear mappings.

2.2 Training

In order to choose which are the best weights for achieving the best performance the DNN must be trained. The procedure consist in taking an input x_i , applying inference (forward propagation) in order to obtain a result y_i .

Then, using the loss function measures how much the prediction differed from the true label $\hat{y}_i$.

Then, thanks to gradient backpropagation (backward propagation), we can measure how much each weight contributed to the final error.

In the end the optimizer updated properly the weights in order to correct the prediction.

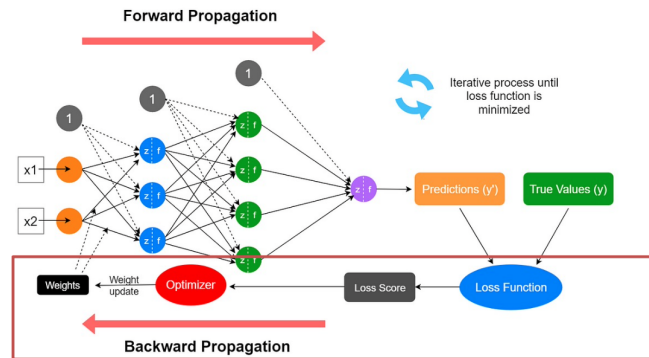


Figure 2: General DNN training loop

Since the gradients are computed with respect to the loss, their exact computation would require averaging over all data points in the dataset, which is impractical for modern large-scale datasets.

To address this, **minibatches** are introduced: subsets of the dataset containing a manageable number of samples that still statistically approximate the overall data distribution.

A complete iteration over all minibatches (thus processing the entire dataset once) is called an **epoch**.

2.3 Backpropagation

Backpropagation is the algorithm used to compute how much each weight in a neural network contributed to the total output error.

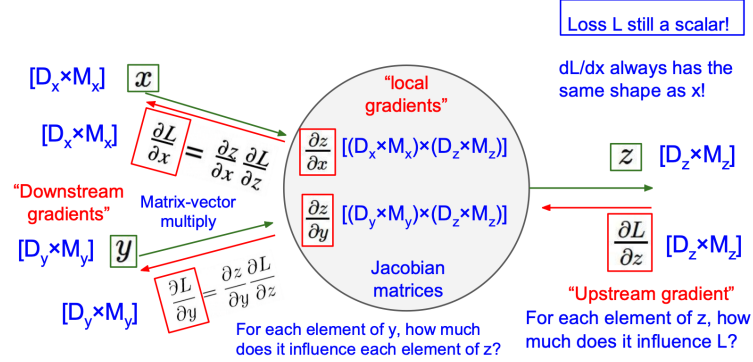


Figure 3: Backpropagation in one perceptron

It provides the gradients of the loss function with respect to all parameters of the model and consists of the following phases:

1. **Forward pass** Each input sample is propagated through the network layer by layer. For each layer $l = 1, 2, \dots, L$:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f^{(l)}(z^{(l)}),$$

where $a^{(0)} = x$ is the input vector. During this step, all intermediate values $z^{(l)}$ and $a^{(l)}$ are stored because they are required later to compute local gradients. The final output $\hat{y} = a^{(L)}$ is compared with the target label y using a loss function $L(\hat{y}, y)$.

2. **Backward pass** The backward phase computes how the loss changes with respect to every parameter by applying the chain rule of calculus in reverse order (from the output layer back to the first hidden layer).
For the output layer ($l = L$):

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} f'^{(L)}(z^{(L)}),$$

where the multiplication is elementwise.

For hidden layers ($l = L - 1, L - 2, \dots, 1$):

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} f'^{(l)}(z^{(l)}).$$

Each $\delta^{(l)}$ represents the local error signal for layer l , showing how sensitive the loss is to changes in its pre-activation values.

3. **Gradient computation and aggregation** Once the $\delta^{(l)}$ values are known for all $l = 1, \dots, L$, the gradients of the loss with respect to the weights and biases are computed as:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

When using minibatches, these gradients are averaged across all samples in the batch:

$$\nabla W^{(l)} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L_i}{\partial W^{(l)}}, \quad l = 1, \dots, L,$$

where m is the batch size.

4. **Optimization step** The optimizer uses the computed gradients to update the parameters for each layer $l = 1, \dots, L$. In the case of gradient descent:

$$W^{(l)} \leftarrow W^{(l)} - \eta \nabla W^{(l)}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \nabla b^{(l)},$$

where η is the learning rate controlling the magnitude of each update. More advanced optimizers (e.g. Adam) adapt η dynamically based on running statistics of past gradients, but they still rely on the same gradients computed via backpropagation.

The learning rate can be made dynamic depending on the training progress, typically measured in **total steps** or epochs. The total number of steps T is given by

$$T = \text{epochs} \times \frac{N}{B},$$

where N is the total number of training samples and B is the minibatch size. Dynamic learning rates can be implemented using learning rate schedulers, which adjust the learning rate as a function of the current step or epoch (e.g. cosine, one-cycle).

3 Convolutional Neural Networks

3.1 Limits of DNNs in computer vision

In the field of computer vision, standard fully connected DNNs exhibit several limitations:

- **Low scalability:** they have a very high number of parameters, which leads to large memory requirements and high computational cost.
- **Ignoring locality:** each neuron is connected to all input pixels, so spatial relationships between nearby pixels are not explicitly leveraged.
- **Prone to overfitting:** the full connectivity allows the network to memorize training data rather than generalize, especially when the dataset is small relative to the number of parameters.

3.2 Definition

Convolutional Neural Networks (CNNs) replace fully connected layers with convolutional layers that use a small, shared kernel (or filter) applied across the entire input image. The kernel slides over the input and, using a correlation-like operation (since it doesn't flip the kernel), produces feature maps that capture local patterns.

This architecture addresses several limitations of fully connected DNNs in computer vision:

- **Reduced number of parameters:** sharing the same kernel across the image drastically reduces the total number of weights, improving scalability and computational efficiency.
- **Locality bias:** the convolution operation focuses only on local neighborhoods, allowing the network to exploit spatial structure and local correlations between pixels.
- **Translation invariance:** since the same kernel is applied across the entire image, the network can recognize patterns regardless of their position, making it more robust to shifts in the input.

3.3 Building blocks

3.3.1 Convolution Layer

A convolutional layer operates on a three-dimensional input tensor $I \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$.

The layer contains a set of C_{out} learnable kernels and C_{out} bias terms.

Then each layer is also characterized by a **stride** S , which defines the step size with which the kernel moves across the input, and a **padding** P , which specifies the number of zero (or constant) values added around the input borders to control the spatial size of

the output feature map.

The output vector $O \in \mathbb{R}^{C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}}$ is computed as:

$$O[c_{\text{out}}, x, y] = b[c_{\text{out}}] + \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{i=0}^{K_H-1} \sum_{j=0}^{K_W-1} K[c_{\text{out}}, c_{\text{in}}, i, j] \cdot I[c_{\text{in}}, Sx + i - P, Sy + j - P].$$

The resulting spatial dimensions are determined by:

$$H_{\text{out}} = \frac{H_{\text{in}} + 2P - K_h}{S} + 1, \quad W_{\text{out}} = \frac{W_{\text{in}} + 2P - K_w}{S} + 1.$$

In general this layer has:

- **#params** = $C_{\text{out}} \times (C_{\text{in}} \times K_H \times K_W + 1)$, where the +1 accounts for the bias term per output channel.
- **#FLOPs** $\approx 2 \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}} \times C_{\text{in}} \times K_H \times K_W$, assuming each multiply-add counts as two floating-point operations.

3.3.2 Batch Normalization Layer

This layer normalizes the input activations so that each feature has approximately zero mean and unit variance, which stabilizes and accelerates training. Formally, given an input mini-batch

$$X = \{x_1, x_2, \dots, x_m\},$$

Batch Normalization operates in two distinct phases:

1. **Training:** For each feature dimension, the layer computes the batch statistics:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The input is then normalized as:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

and scaled and shifted using learnable parameters γ (scale) and β (shift):

$$y_i = \gamma \hat{x}_i + \beta.$$

Running estimates of μ_B and σ_B^2 are also updated to be used during inference.

2. **Inference:** The normalization is performed using the running mean μ_{run} and variance σ_{run}^2 accumulated during training:

$$\hat{x} = \frac{x - \mu_{\text{run}}}{\sqrt{\sigma_{\text{run}}^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta.$$

PS: ϵ is a small constant added for numerical stability.

In general this layer has:

- **#params** = $2 \times C$, where C is the number of feature channels. The two parameters are the learnable scale γ and shift β applied to each channel.
- **#FLOPs** $\approx 4 \times C \times H \times W$, accounting for the normalization (subtraction and division) and the subsequent affine transformation (multiplication by γ and addition of β) for each element in the feature map.

3.3.3 Pooling Layer

This layer reduces the spatial resolution of each channel independently, decreasing the number of parameters and increasing translation invariance and robustness to noise. Pooling operates over small non-overlapping regions of the input feature map, typically of size 2×2 or 3×3 . Such sizes are chosen because, when combined with a stride equal to the window size (e.g., stride $S = 2$ for a 2×2 window), they effectively downsample the feature map by a factor of two in each spatial dimension, preserving the most relevant spatial information while reducing computational cost.

The reduction is performed by applying a fixed aggregation function over each region. The most common types are:

- **Max Pooling:** Selects the maximum value within each local region:

$$Y[i, j] = \max_{(m, n) \in \Omega(i, j)} X[m, n],$$

where $\Omega(i, j)$ denotes the spatial region of the input corresponding to the pooling window centered at (i, j) .

- **Average Pooling:** Computes the average value within each region:

$$Y[i, j] = \frac{1}{|\Omega(i, j)|} \sum_{(m, n) \in \Omega(i, j)} X[m, n].$$

In general this layer has:

- **#params** = 0, since pooling layers contain no learnable parameters. The operation is purely functional and applied independently to each channel.
- **#FLOPs** $\approx C \times H_{\text{out}} \times W_{\text{out}} \times K_H \times K_W$, where each output element requires scanning the corresponding pooling region of size $K_H \times K_W$ in the input to compute the maximum or average.

3.3.4 Fully Connected Layer

A Fully Connected (FC) layer, also called a Dense layer, connects every input neuron to every output neuron. It operates on a one-dimensional input vector and produces a one-dimensional output vector, performing a global linear transformation.

Formally, given an input vector $x \in \mathbb{R}^{N_{\text{in}}}$, the layer computes the output like a linear classifier.

In convolutional neural networks, the input tensor $X \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ is first flattened into a vector of length $N_{\text{in}} = C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}$ before being processed by the FC layer. The number of output neurons N_{out} corresponds to the desired output dimensionality (the number of classes in a classification task).

In general this layer has:

- **#params** = $N_{\text{out}} \times (N_{\text{in}} + 1)$, where the +1 accounts for the bias term associated with each output neuron.
- **#FLOPs** $\approx 2 \times N_{\text{in}} \times N_{\text{out}}$, since each output requires N_{in} multiplications and $N_{\text{in}} - 1$ additions (counting one multiply-add as two floating-point operations).

3.4 Separable Convolutions

Thanks to properties of convolutions, we can *approximate* a standard convolution operation with more efficient factorized versions. However, these decompositions **do change the structure and expressive power** of the layer, creating different hypothesis spaces rather than just computationally efficient equivalents.

There's an important trade-off: while separable convolutions dramatically reduce computational cost and parameters, they *constrain the space of learnable functions* compared to standard convolutions.

3.4.1 Spatially Separable Convolutions

This method factorizes a 2D kernel into two 1D kernels.

- **Operation:** Decomposes a $K \times K$ kernel into a $K \times 1$ kernel followed by a $1 \times K$ kernel.
- **Parameters:**
 - **Standard Conv:** K^2 parameters (per channel)
 - **Spatially Separable:** $K + K = 2K$ parameters
- **Computational Cost:**
 - **Standard Conv:** $H_o \cdot W_o \cdot K^2$ operations
 - **Spatially Separable:** $H_o \cdot W_o \cdot (K + K)$ operations
- **Limitation:** Not all 2D kernels are spatially separable (only rank-1 kernels), limiting its general use.

3.4.2 Depthwise Separable Convolutions

This is the most widely used type, decomposing a standard convolution into a **depthwise** and a **pointwise** convolution.

- **Operation:**
 1. **Depthwise Conv:** Applies C_i separate $K \times K \times 1$ kernels (one per input channel).
 2. **Pointwise Conv:** Applies a $1 \times 1 \times C_i \times C_o$ kernel to mix channels.
- **Parameters:**
 - **Standard Conv:** $K^2 \cdot C_i \cdot C_o$
 - **Depthwise Separable:** $(K^2 \cdot C_i) + (1 \cdot 1 \cdot C_i \cdot C_o)$
- **Computational Cost:**
 - **Standard Conv:** $H_o \cdot W_o \cdot K^2 \cdot C_i \cdot C_o$
 - **Depthwise Separable:** $H_o \cdot W_o \cdot (K^2 \cdot C_i + C_i \cdot C_o)$

3.4.3 Parameter Reduction Ratio

The efficiency gain for Depthwise Separable Convolutions is substantial. The ratio of its cost to a standard convolution is approximately:

$$\begin{aligned} \frac{\text{Cost}_{\text{sep}}}{\text{Cost}_{\text{std}}} &= \frac{H_o \cdot W_o \cdot (K^2 \cdot C_i + C_i \cdot C_o)}{H_o \cdot W_o \cdot K^2 \cdot C_i \cdot C_o} \\ &= \frac{K^2 \cdot C_i + C_i \cdot C_o}{K^2 \cdot C_i \cdot C_o} \\ &= \frac{K^2 \cdot C_i}{K^2 \cdot C_i \cdot C_o} + \frac{C_i \cdot C_o}{K^2 \cdot C_i \cdot C_o} \\ &= \frac{1}{C_o} + \frac{1}{K^2} \end{aligned}$$

For a typical 3×3 convolution with many output channels, this results in an **8x to 9x reduction** in both parameters and computations.

4 Quantization & Pruning

We want to make neural networks faster and lighter (both in memory and computation) without sacrificing too much accuracy. The idea is to do that by optimizing how and what numbers are represented in the NNs.

4.1 Numeric Data Types

4.1.1 Integer

- **unsigned integer:**

given n -bit the number range is $[0, 2^n - 1]$

$$10110110_2 \rightarrow 1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 \rightarrow 182_{10}$$

- **signed integer:**

- **sign-magnitude representation:**

Given n -bit the number range is $[-(2^n - 1), 2^n - 1]$.

One bit (the MSB) represents the sign, while the remaining $n - 1$ bits represent the magnitude.

Zero has two representations: positive (00..0) and negative (10..0).

$$\begin{aligned} 0\ 010110110_2 &\rightarrow +182_{10} \\ 1\ 010110110_2 &\rightarrow -182_{10} \end{aligned}$$

- **two's complement representation:** Given n -bit the number range is $[-2^n, 2^n - 1]$.

Negative numbers are represented by inverting all bits of the positive number and adding 1.

In this way there is only one representation for zero and arithmetic operations (addition, subtraction) are identical for signed and unsigned integers at the hardware level.

$$\begin{aligned} 010110110 &\rightarrow +182_{10} \text{ (same as unsigned integer)} \\ 182_2 = 010110110_2 &\rightarrow 101001001_2 + 1 \rightarrow 101001010_2 = -182_2 \end{aligned}$$

4.1.2 Fixed-Point Number

It is a numeric representation where a fixed number of bits is allocated to the integer part and a fixed number of bits to the fractional part. Unlike floating-point numbers,

the position of the radix (decimal) point is constant, which allows for simpler and faster hardware implementations but limits dynamic range.

It benefits from the deterministic precision and performance but suffers the limited dynamic range and so the poor flexibility for high and small values.

```
0101101100000000002 → (0)(10110110).(00000000)

(0) → +
(101101102) → (18210)
(000000002) → 0 · 2-1 + 0 · 2-2 + ... + 0 · 2-7 + 0 · 2-8 = 010

= +182.0
```

4.1.3 Floating-Point Number

As the fixed-point number, also this has 3 components: sign, exponent (range), fraction (precision/mantissa). In order to obtain the base-10 value the applied rule is :

$$(-1)^S \cdot (1 + F) \cdot 2^{(E-B)}$$

Where B , the bias, is needed since the exponent can only be positive and so we need it to represent small values that require negative exponent. Given n bits in the exponent, the bias is $2^{n-1} - 1$.

This structure allows not to set the point statically but dynamically (thanks to the exponent). This allows the same number of significant digits to represent values across an enormous range, from microscopic to astronomical scales.

```
Considering 32-bit with 8-bit exponent, so with bias 127:
010000110011011000000000000000002
⇒ (0)(10000110)(011011000000000000000000)2

(0) → +
(100001102) → 13410
(0110110000000000000000002) → 0.42187510

(+) (1 + 0.421875) · 2134-127 = 1.421875 · 27 = +182
```

4.2 Quantization

This reduces the numerical precision of the model's parameters and activations. Instead of using high-precision floating-point values (like 32-bit floats), the network uses smaller

representations (like 8-bit integers), counteracting the cons of being static with pre-computation and calibration that makes the fixed precision choices data-informed and near-optimal.

In DNN we have two variables to consider:

- **weights:** variable in training but fixed at inference time, so the range is known.
- **activations:** variable also at testing time but dependant on inputs, so they can be evaluated statistically with calibration.

With this procedure, we can achieve significant efficiency gains across the system:

- **Reduced Memory Footprint:** Transitioning from 32-bit floats to 8-bit integers immediately reduces the model's RAM and ROM occupation by approximately 75% ($8/32 = 1/4$ of the original size).
- **Increased Data-Transfer Bandwidth:** When loading data, the same hardware instruction can now move 4 times more values (e.g., four 8-bit integers instead of one 32-bit float), effectively quadrupling the memory bandwidth.
- **Optimized Computations:** By converting the problem to integer arithmetic, the CPU can utilize its faster and more power-efficient Arithmetic Logic Units (ALUs) instead of the Floating-Point Units (FPUs). This leads to substantial speed gains and longer battery life. Furthermore, modern SIMD (Single Instruction, Multiple Data) units can perform up to 4 times more operations per cycle on 8-bit integers compared to 32-bit floats, further accelerating parallel computations.

4.2.1 Linear Quantization

It is a technique to map high-precision floating-point numbers (like FP32) to low-precision integers (like INT8). It's called "linear" (or more precisely, "affine") because the mapping from integers to real numbers follows a straight-line equation.

$$r = (q - Z) \cdot S$$

Where:

- r is the real, floating-point value
- q is the quantized integer value corresponding to r
- Z is the Zero Point: an integer value that ensures the real number 0 can be represented exactly by a quantized value. This is crucial for layers like ReLU that produce zeros.
- S is the scale: a floating-point value that defines the step size between each integer value. It scales the integer range to the floating-point range.

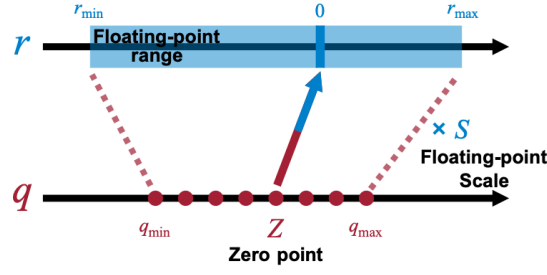


Figure 4: Linear quantization visualization

In order to better perform the quantization we have to calibrate the scale and the zero point. According to the problem as represented in Fig.??, we can estimate them:

- Scale:

$$\begin{cases} r_{\max} = S(q_{\max} - Z) \\ r_{\min} = S(q_{\min} - Z) \end{cases}$$

$$r_{\max} - r_{\min} = S(q_{\max} - Z) - S(q_{\min} - Z) = S(q_{\max} - q_{\min})$$

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

- Zero point:

$$r_{\min} = S(q_{\min} - Z)$$

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

$$Z = \text{round}\left(q_{\min} - \frac{r_{\min}}{S}\right)$$

mybox **NB: there are special cases**

For weights, that usually are symmetrically distributed around 0, $Z = 0$.

For activations, that usually (because of RELUs) are strongly imbalanced with respect to 0, it is likely to see $Z \neq 0$.

In practice, when we apply the formula to and activation **a** or to the weights **w**, given N -bits the formulas can be seen as:

	Scale (S)	Zero-point (Z)
Activations	$S = \frac{a_{\max} - a_{\min}}{2^N - 1}$	$Z = \frac{a_{\max} - a_{\min}}{2}$
Weights	$S = \frac{2 \cdot \max(w_{\max} , w_{\min})}{2^N - 1}$	$Z = 0$

Notice that they are simplifications:

- **In activations and weights** $q_{\max} - q_{\min} = 2^N - 1$
This is exact and comes from integer representation: with N bits we have 2^N discrete values, but the number of steps between them is $2^N - 1$. For signed integers in two's complement: $q_{\max} = 2^{N-1} - 1$, $q_{\min} = -2^{N-1}$, so $q_{\max} - q_{\min} = 2^N - 1$.
- **In activations** $Z \approx \frac{r_{\max} - r_{\min}}{2}$
...mean of the activations...why?
- **In weights** $r_{\max} - r_{\min} \approx 2 \cdot \max(|r_{\max}|, |r_{\min}|)$
This simplification forces symmetric quantization around zero. Instead of using the actual range $[r_{\min}, r_{\max}]$, we create a symmetric range $[-\alpha, +\alpha]$ where $\alpha = \max(|r_{\max}|, |r_{\min}|)$. This ensures the range is centered at zero and utilizes the full integer range efficiently for symmetric weight distributions.
- **In weights** $Z = 0$ as explained before.
This is a design choice for computational efficiency. With symmetric weight distributions centered around zero, setting $Z = 0$ eliminates the need for zero-point adjustments in weight-related computations, simplifying the hardware implementation and improving performance.

Then, it is important to remark that for activations we can only estimate the ranges, so after computing the parameters it is necessary another saturation step to ensure that the values fall in the representable range.

4.2.2 Fixed Precision vs Mixed Precision

- **Fixed precision**
We choose N bits for quantizing all the network. This has some important problems as not all the layers need quantization in the same way and so we face a tradeoff: a large range (high N) to preserve accuracy risks being overly inclusive and wastes memory/compute, while a small range (low N) to improve efficiency risks being too exclusive, clipping important values and degrading accuracy.
- **Mixed Precision**
Different bit-widths are assigned to different layers or even channels through automated search algorithms (e.g., greedy methods, simulated annealing). This allows precision to be tailored layer-wise, using higher bit-widths for sensitive layers and lower bit-widths for robust ones, achieving optimal trade-offs between model size, computational efficiency, and accuracy.

mybox NOTE: In both cases the N -width can be different in the same layer between activations and weights. Moreover in both cases we can change to vary S, Z across the channels of the same layer.

4.2.3 Binary Neural Networks

They are an extreme form of neural network quantization where both weights and activations are constrained to only two values, -1 (represented by logic-0) and $+1$ (represented by logic-1).

This approach achieves massive memory savings ($32\times$ reduction) by using 1 bit per value instead of 32 bits. More importantly, it enables radical computational optimizations by replacing traditional Multiply-Accumulate (MAC) operations:

- **Multiplications are replaced by XNOR operations**

In binary networks, multiplying two values $\{-1, +1\}$ is equivalent to an XNOR gate when using the logic mapping ($-1 \rightarrow 0, +1 \rightarrow 1$). The truth table shows this equivalence:

A	B	$A \times B$	Logic A	Logic B	XNOR
-1	-1	$+1$	0	0	1
-1	$+1$	-1	0	1	0
$+1$	-1	-1	1	0	0
$+1$	$+1$	$+1$	1	1	1

This transformation allows replacing floating-point multiplications with single-bit XNOR operations, which are orders of magnitude faster and more energy-efficient.

- **Additions/accumulations are replaced by Popcount**

Accumulating binary products requires counting the occurrences of $+1$ and -1 results. Let p = count of $+1$ s (XNOR=1) and n = count of -1 s (XNOR=0), with $N = p + n$ (N represents the total number of elements being processed in a given operation, namely the length of the vectors being multiplied and accumulated):

$$\text{Accum} = p \cdot (+1) + n \cdot (-1) = p - n = p - (N - p) = 2p - N$$

Since 2 and N are constants, the accumulation is equivalent to counting the number of 1s (popcount). Modern processors can perform popcount operations extremely efficiently, often in a single instruction, making accumulation much faster than traditional addition.

It is important to highlight that the significant footprint reductions and the computational speed-ups have to face the significant drop in accuracy scores.

4.2.4 Non-Uniform quantization

The key idea is: since weight/activation distributions are rarely uniform, why use uniformly spaced quantization points?

So we can explore two main approaches:

- **Log-domain quantization**

Most weights/activations cluster near zero (linear quantization wastes levels on larger values that rarely occur), so we want more quantization levels there. With a logarithmic scale we can inspect negative orders of magnitude.

- **Weight clustering** (or Learned quantization)

Instead of reducing bit-width per value, reduce the number of distinct values. Practically it:

1. Groups similar weights into clusters (k-means) and gets their centroids.
2. Stores only the centroids + cluster indexes. In this way we store a matrix of integers (indexes) and a list of floats (centroids). Saving much more memory instead of storing a matrix of floats.
3. When required (inference time), the list of centroids will work as a lookup table, retrieving the float value for the index in a certain cell of the matrix.

4.2.5 Strategies

We can so divide those quantization methods into two general principles:

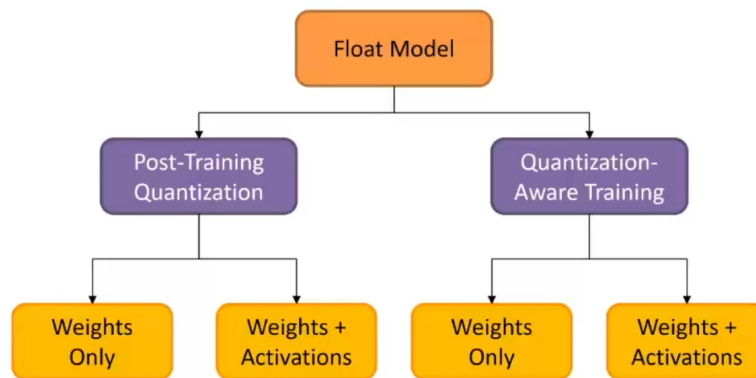


Figure 5: Quantization strategies schema

- **Post-Training Quantization**

We train the network, obtain the weights (float) and then, applying a rule/formula we transform them in integers. We don't retrain the network. This methods are preferred as training is very expensive.

- **Quantize only weights**

More common as they are more stable and so you can work better on them after training as they are fixed. The problem is that we have to face operations between two different types of numbers. Typically, we unpack integers into floats for the computation and continue passing floats through the network. This is of course more expensive and minimize the gain from quantization.

Another solution is to quantize the floats on the fly which is more likely to drop accuracies (computing S/Z on an activation during inference is statistically risky), but we gain in efficiency as we work entirely with quantized data.

- **Quantize both weights and activations**

We quantize the activations considering a representative subset of inputs. Typically small amounts of them is sufficient to compute a useful statistics, considering that outliers are handled by saturation.

- **Quantization-Aware Training**

Those methods enable the network, during training, to also learn which is the best quantization. In this way we can achieve better accuracies but of course it is more expensive and so better for smaller networks.

There are issues in backpropagating the gradient of the re-quantization function. During backpropagation we have to round the gradient, but in this way small and useful updates disappears and so training become harder to converge.

4.3 Pruning

You compute a network with a lot of weights (different from 0) and then, minimizing the error during the process, you try to set as many of them as possible to 0. This comes from the biological intuition that our best brain capacity doesn't coincide with the moment we have the most synapses. It is a form a compression, as setting weights to 0 is like removing them.

4.3.1 Strategies

- Weight Saliency Pruning (not used)

- **Magnitude Based Pruning**

The idea is that we set weights to 0 if they are smaller than a certain amount. I only have to decide a threshold. After the pruning there is a training part as we have to adjust the non-zero weights in order to compensate the changes (to do that we have to freeze the 0 weights). Today networks are very sensitive to pruning. We can also decide to schedule the pruning in order to decide the speed rate of the pruning. Like in learning rate schedulers, starting low and then gradually increasing can lead to better performances.

4.3.2 Limitations

Now we are setting to zero weights without knowing where they are. This can cause the weight matrices to have areas with many zeros and areas without any of them. This is **unstructured pruning**. The problem is that this irregularity makes it difficult for the hardware to efficiently work on them (e.g. caches, parallel

operations).

We can solve this by using **block-based pruning**: we decide the block size and then we enforce a number of weights to zero per each block.

In this way the sparsity is very regular and i can compress the data structure in a very efficient way: store the values in the reduced matrix and then store the indexes of the original matrix where there were 0s.